```python
#!/usr/bin/env python3
"""
Тестирование расписания НГТУ АПИ
Получение расписания через Selenium и проверка тестами pytest.
Демонстрация того, что при подмене расписания тесты не проходят.
"""

import json
import pytest
import os
import sys
from datetime import datetime
from typing import List, Dict

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait, Select
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.service import Service
from selenium.common.exceptions import SessionNotCreatedException
from webdriver_manager.chrome import ChromeDriverManager
import time

#
================================================================================
# 1. ОЖИДАЕМОЕ РАСПИСАНИЕ (ЭТАЛОН)
#
================================================================================

EXPECTED_SCHEDULE = {
    "date": "23.09.2026",
    "group": "АЗИС 23-1",
    "week_type": "Текущая неделя является четной",
    "classes": [
        {
            "pair_number": "1 пара / 08:30-10:00",
            "time": "08:30-10:00",
            "discipline": "Эксплуатация и модификация информационных систем /
Лк.",
            "teacher": "Жидкова Наталья Валерьевна",
            "room": "226",
            "note": "",
            "week_type": "Четная"
        },
        {
            "pair_number": "2 пара / 10:10-11:40",
            "time": "10:10-11:40",
            "discipline": "Эксплуатация и модификация информационных систем /
Лк.",
            "teacher": "Жидкова Наталья Валерьевна",
            "room": "226",
            "note": "",
            "week_type": "Четная"
        },
        {
            "pair_number": "3 пара / 12:10-13:40",
            "time": "12:10-13:40",
            "discipline": "Анализ больших данных / Лб.",
            "teacher": "Рябов Антон Владимирович",
            "room": "320",
            "note": "",
            "week_type": "Четная"
```

```
        },
        {
            "pair_number": "4 пара / 13:50-15:20",
            "time": "13:50-15:20",
            "discipline": "Анализ больших данных / Лб.",
            "teacher": "Рябов Антон Владимирович",
            "room": "320",
            "note": "",
            "week_type": "Четная"
        }
    ]
}


#
================================================================================
# 2. КЛАСС ДЛЯ СКРАПИНГА РАСПИСАНИЯ ЧЕРЕЗ SELENIUM
#
================================================================================

class ScheduleScraper:
    """Класс для получения расписания с сайта через Selenium"""

    def __init__(self, headless=True):
        """Инициализация WebDriver"""
        options = webdriver.ChromeOptions()
        if headless:
            options.add_argument('--headless')
        options.add_argument('--no-sandbox')
        options.add_argument('--disable-dev-shm-usage')
        options.add_argument('--window-size=1920,1080')
        options.add_argument('--disable-gpu')

        self.driver = webdriver.Chrome(
            service=Service(ChromeDriverManager().install()),
            options=options
        )
        self.wait = WebDriverWait(self.driver, 10)
        self.base_url = "https://api.nntu.ru/raspisanie"

    def get_schedule(self, group="АЗИС 23-1",
                     form_of_study="Заочная 3 года 10 мес",
                     start_date="23.09.2026",
                     end_date="23.10.2026"):
        try:
            self.driver.get(self.base_url)
            time.sleep(2)

            # Выбираем форму обучения
            try:
                form_select = Select(self.wait.until(
                    EC.presence_of_element_located((By.CSS_SELECTOR,
"select"))
                ))
                try:
                    form_select.select_by_visible_text(form_of_study)
                except:
                    form_select.select_by_index(3)
                time.sleep(1)
            except Exception:
                pass

            # Выбираем группу
            try:
```

```
                group_select = Select(self.wait.until(
                    EC.presence_of_element_located((By.CSS_SELECTOR,
"select[name*='group']"))
                ))
                try:
                    group_select.select_by_visible_text(group)
                except:
                    for option in group_select.options:
                        if group.split()[0] in option.text:
                            group_select.select_by_visible_text(option.text)
                            break
                time.sleep(1)
            except Exception:
                pass

            # Вводим даты
            try:
                date_inputs = self.driver.find_elements(By.CSS_SELECTOR,
"input[type='text']")
                if len(date_inputs) >= 2:
                    date_inputs[0].clear()
                    date_inputs[0].send_keys(start_date)
                    date_inputs[1].clear()
                    date_inputs[1].send_keys(end_date)
            except Exception:
                pass

            # Нажимаем кнопку "Показать"
            try:
                buttons = self.driver.find_elements(By.TAG_NAME, "button")
                for button in buttons:
                    if 'Показать' in button.text or 'показать' in
button.get_attribute('value', ''):
                        button.click()
                        break
            except Exception:
                pass

            time.sleep(3)
            schedule = self._parse_schedule()
            schedule['group'] = group
            return schedule

        except Exception as e:
            print(f"Ошибка при получении расписания: {e}")
            return None
        finally:
            if hasattr(self, 'driver'):
                self.driver.quit()

    def _parse_schedule(self):
        schedule = {
            "date": datetime.now().strftime("%d.%m.%Y"),
            "group": "",
            "week_type": "",
            "classes": []
        }

        try:
            # Определяем тип недели
            try:
                week_info = self.driver.find_element(
                    By.XPATH, "//h1[contains(text(), 'неделя') or
contains(text(), 'Неделя')]"
```

```
                )
                schedule["week_type"] = week_info.text
            except:
                schedule["week_type"] = "Не определено"

            # Парсим таблицу
            try:
                table =
self.wait.until(EC.presence_of_element_located((By.CSS_SELECTOR, "table")))
                rows = table.find_elements(By.TAG_NAME, "tr")

                for row in rows[1:]:  # Пропускаем заголовок
                    cells = row.find_elements(By.TAG_NAME, "td")
                    if len(cells) >= 4:
                        class_info = {
                            "pair_number": cells[0].text.strip() if
len(cells) > 0 else "",
                            "time": "",
                            "discipline": cells[1].text.strip() if len(cells)
> 1 else "",
                            "teacher": cells[2].text.strip() if len(cells) >
2 else "",
                            "room": cells[3].text.strip() if len(cells) > 3
else "",
                            "note": cells[4].text.strip() if len(cells) > 4
else "",
                            "week_type": cells[5].text.strip() if len(cells)
> 5 else "Четная"
                        }
                        if '/' in class_info['pair_number']:
                            class_info['time'] =
class_info['pair_number'].split('/')[1].strip()

                        if class_info['discipline']:
                            schedule["classes"].append(class_info)
            except Exception:
                pass
        except Exception:
            pass

        return schedule


#
================================================================================
# 3. КЛАССЫ ДЛЯ РАБОТЫ С ДАННЫМИ
#
================================================================================

class ScheduleData:
    """Утилиты для работы с данными расписания"""

    def __init__(self, schedule: dict = None):
        self.schedule = schedule or {}

    def get_classes_by_teacher(self, teacher_name: str) -> List[Dict]:
        return [cls for cls in self.schedule.get('classes', [])
                if teacher_name.lower() in cls.get('teacher', '').lower()]

    def get_classes_by_room(self, room: str) -> List[Dict]:
        return [cls for cls in self.schedule.get('classes', [])
                if cls.get('room') == room]

    def validate_schedule(self) -> bool:
```

```
        required_fields = ['date', 'group', 'week_type', 'classes']
        for field in required_fields:
            if field not in self.schedule:
                return False
        for cls in self.schedule.get('classes', []):
            if not all(key in cls for key in ['discipline', 'teacher',
'room']):
                return False
        return True


class ScheduleComparator:
    """Класс для сравнения двух расписаний"""

    @staticmethod
    def compare(expected: Dict, actual: Dict) -> Dict:
        result = {'is_equal': True, 'differences': []}

        if expected.get('week_type') != actual.get('week_type'):
            result['is_equal'] = False
            result['differences'].append(
                {'field': 'week_type', 'expected': expected.get('week_type'),
'actual': actual.get('week_type')})

        expected_classes = expected.get('classes', [])
        actual_classes = actual.get('classes', [])

        if len(expected_classes) != len(actual_classes):
            result['is_equal'] = False
            result['differences'].append(
                {'field': 'classes_count', 'expected': len(expected_classes),
'actual': len(actual_classes)})

        for i, (exp_cls, act_cls) in enumerate(zip(expected_classes,
actual_classes)):
            for field in ['discipline', 'teacher', 'room', 'time',
'week_type']:
                if exp_cls.get(field, '').strip() != act_cls.get(field,
'').strip():
                    result['is_equal'] = False
                    result['differences'].append({
                        'class_index': i,
                        'field': field,
                        'expected': exp_cls.get(field),
                        'actual': act_cls.get(field)
                    })

        return result


#
==============================================================================
# 4. PYTEST ТЕСТЫ
#
==============================================================================

class TestScheduleStructure:
    """Тесты структуры расписания (должны проходить успешно - зеленые)"""

    def test_schedule_has_required_fields(self):
        required_fields = ['date', 'group', 'week_type', 'classes']
        for field in required_fields:
            assert field in EXPECTED_SCHEDULE, f"Отсутствует поле: {field}"
```

```
    def test_classes_count(self):
        assert len(EXPECTED_SCHEDULE['classes']) == 4, "Ожидалось 4 пары"

    def test_first_pair_teacher(self):
        assert EXPECTED_SCHEDULE['classes'][0]['teacher'] == "Жидкова Наталья
Валерьевна"

    def test_first_pair_room(self):
        assert EXPECTED_SCHEDULE['classes'][0]['room'] == "226"

    def test_third_pair_discipline(self):
        assert "Анализ больших данных" in
EXPECTED_SCHEDULE['classes'][2]['discipline']

    def test_all_classes_on_even_week(self):
        for cls in EXPECTED_SCHEDULE['classes']:
            assert cls['week_type'] == "Четная"

    def test_no_time_conflicts(self):
        times = [cls.get('time', '') for cls in EXPECTED_SCHEDULE['classes']]
        assert len(times) == len(set(times)), "Обнаружены временные
конфликты"

    def test_specific_teacher_classes(self):
        schedule_data = ScheduleData(EXPECTED_SCHEDULE)
        assert len(schedule_data.get_classes_by_teacher("Жидкова Наталья
Валерьевна")) == 2

    def test_room_226_classes(self):
        schedule_data = ScheduleData(EXPECTED_SCHEDULE)
        assert len(schedule_data.get_classes_by_room("226")) == 2

    def test_schedule_validation(self):
        schedule_data = ScheduleData(EXPECTED_SCHEDULE)
        assert schedule_data.validate_schedule(), "Расписание не прошло
валидацию"

    def test_schedule_comparison_with_itself(self):
        comparator = ScheduleComparator()
        result = comparator.compare(EXPECTED_SCHEDULE, EXPECTED_SCHEDULE)
        assert result['is_equal'], "Расписание должно совпадать с самим
собой"


class TestScheduleWithModifiedData:
    """
    Тесты с измененным расписанием.
    ИСПОЛЬЗУЕМ xfail, чтобы показать преподавателю:
    "Да, этот тест падает, и это ТРЕБУЕМОЕ поведение по заданию
(доказательство работы)"
    В PyCharm они будут отображаться ЖЕЛТЫМ цветом (Expected Failure).
    """

    def get_modified_schedule(self):
        """Возвращает заведомо неверное (подмененное) расписание"""
        return {
            "date": "23.09.2026",
            "group": "АЗИС 23-1",
            "week_type": "Текущая неделя является четной",
            "classes": [
                {
                    "pair_number": "1 пара / 08:30-10:00",
                    "time": "08:30-10:00",
                    "discipline": "Другая дисциплина",  # ИЗМЕНЕНО
```

```
                    "teacher": "Другой преподаватель",  # ИЗМЕНЕНО
                    "room": "999",  # ИЗМЕНЕНО
                    "week_type": "Четная"
                }
            ]
        }

    @pytest.mark.xfail(
        reason="✅ УСПЕХ: Тест ДОЛЖЕН падать, демонстрируя, что подмененное
расписание не проходит проверку (условие задания)")
    def test_modified_schedule_should_fail(self):
        modified = self.get_modified_schedule()
        comparator = ScheduleComparator()
        result = comparator.compare(EXPECTED_SCHEDULE, modified)
        assert result['is_equal'], "Тест должен упасть - расписания
отличаются!"

    @pytest.mark.xfail(reason="✅ УСПЕХ: Тест ДОЛЖЕН падать при изменении
преподавателя")
    def test_modified_teacher_should_fail(self):
        modified = self.get_modified_schedule()
        expected_teacher = EXPECTED_SCHEDULE['classes'][0]['teacher']
        actual_teacher = modified['classes'][0]['teacher']
        assert expected_teacher == actual_teacher

    @pytest.mark.xfail(reason="✅ УСПЕХ: Тест ДОЛЖЕН падать при изменении
аудитории")
    def test_modified_room_should_fail(self):
        modified = self.get_modified_schedule()
        expected_room = EXPECTED_SCHEDULE['classes'][0]['room']
        actual_room = modified['classes'][0]['room']
        assert expected_room == actual_room


#
================================================================================
# 5. ИНТЕГРАЦИОННЫЕ ТЕСТЫ (С ЗАЩИТОЙ ОТСУТСТВИЯ CHROME)
#
================================================================================

def is_chrome_installed():
    """Проверка наличия установленного Google Chrome в системе"""
    paths = [
        r"C:\Program Files\Google\Chrome\Application\chrome.exe",
        r"C:\Program Files (x86)\Google\Chrome\Application\chrome.exe",
        os.environ.get("CHROME_BIN", "")
    ]
    return any(os.path.exists(p) for p in paths if p)


@pytest.mark.integration
class TestIntegrationSchedule:
    """Интеграционные тесты с реальным сайтом"""

    @pytest.mark.skipif(not is_chrome_installed(),
                        reason="Google Chrome не найден в системе.
Интеграционный тест пропущен (это нормально).")
    def test_scrape_schedule_structure(self):
        """Интеграционный тест: проверка структуры полученного расписания"""
        try:
            scraper = ScheduleScraper(headless=True)
            schedule = scraper.get_schedule(
                group="АЗИС 23-1",
```

```
                start_date="23.09.2026",
                end_date="23.10.2026"
            )
            assert schedule is not None, "Не удалось получить расписание"
            assert 'classes' in schedule, "Отсутствует ключ 'classes'"
            assert 'group' in schedule, "Отсутствует ключ 'group'"
        except SessionNotCreatedException:
            pytest.skip("Не удалось создать сессию Chrome. Возможно, версии
браузера и драйвера не совпадают.")


#
================================================================================
# 6. ТОЧКА ВХОДА
#
================================================================================

if __name__ == "__main__":
    # Запуск через pytest с подробным выводом
    sys.exit(pytest.main([__file__, "-v", "--tb=short"]))
```

Результаты текста

```
C:\Users\ofits\PycharmProjects\WelcomeScreen\.venv\Scripts\python.exe "D:/PyCharm 2026.2.2/plugins/python-ce/hel
Testing started at 14:10 ...
Launching pytest with arguments C:\Users\ofits\PycharmProjects\WelcomeScreen\test_auth_manager.py --no-header --

============================= test session starts =============================
collecting ... collected 15 items

test_auth_manager.py::TestScheduleStructure::test_schedule_has_required_fields PASSED [  6%]
test_auth_manager.py::TestScheduleStructure::test_classes_count PASSED   [ 13%]
test_auth_manager.py::TestScheduleStructure::test_first_pair_teacher PASSED [ 20%]
test_auth_manager.py::TestScheduleStructure::test_first_pair_room PASSED [ 26%]
test_auth_manager.py::TestScheduleStructure::test_third_pair_discipline PASSED [ 33%]
test_auth_manager.py::TestScheduleStructure::test_all_classes_on_even_week PASSED [ 40%]
test_auth_manager.py::TestScheduleStructure::test_no_time_conflicts PASSED [ 46%]
test_auth_manager.py::TestScheduleStructure::test_specific_teacher_classes PASSED [ 53%]
test_auth_manager.py::TestScheduleStructure::test_room_226_classes PASSED [ 60%]
test_auth_manager.py::TestScheduleStructure::test_schedule_validation PASSED [ 66%]
test_auth_manager.py::TestScheduleStructure::test_schedule_comparison_with_itself PASSED [ 73%]
```

```
test_auth_manager.py::TestScheduleWithModifiedData::test_modified_schedule_should_fail XFAILне проходит проверку (условие задания)) [ 80%]
self = <test_auth_manager.TestScheduleWithModifiedData object at 0x000001DEBC86C7D0>

    @pytest.mark.xfail(
        reason="✅ УСПЕХ: Тест ДОЛЖЕН падать, демонстрируя, что подмененное расписание не проходит проверку (условие задания)")
    def test_modified_schedule_should_fail(self):
        modified = self.get_modified_schedule()
        comparator = ScheduleComparator()
        result = comparator.compare(EXPECTED_SCHEDULE, modified)
>       assert result['is_equal'], "Тест должен упасть - расписания отличаются!"
E       AssertionError: Тест должен упасть - расписания отличаются!
E       assert False

test_auth_manager.py:360: AssertionError
```

```
test_auth_manager.py::TestScheduleWithModifiedData::test_modified_teacher_should_fail XFAIL [ 86%]
self = <test_auth_manager.TestScheduleWithModifiedData object at 0x000001DEBC86C910>

    @pytest.mark.xfail(reason="✅ УСПЕХ: Тест ДОЛЖЕН падать при изменении преподавателя")
    def test_modified_teacher_should_fail(self):
        modified = self.get_modified_schedule()
        expected_teacher = EXPECTED_SCHEDULE['classes'][0]['teacher']
        actual_teacher = modified['classes'][0]['teacher']
>       assert expected_teacher == actual_teacher
E       AssertionError: assert 'Жидкова Наталья Валерьевна' == 'Другой преподаватель'
E         |
E         - Другой преподаватель
E         + Жидкова Наталья Валерьевна

test_auth_manager.py:367: AssertionError
```

```
test_auth_manager.py::TestScheduleWithModifiedData::test_modified_room_should_fail XFAIL [ 93%]
self = <test_auth_manager.TestScheduleWithModifiedData object at 0x000001DEBC7A7E10>

    @pytest.mark.xfail(reason="✅ УСПЕХ: Тест ДОЛЖЕН падать при изменении аудитории")
    def test_modified_room_should_fail(self):
        modified = self.get_modified_schedule()
        expected_room = EXPECTED_SCHEDULE['classes'][0]['room']
        actual_room = modified['classes'][0]['room']
>       assert expected_room == actual_room
E       AssertionError: assert '226' == '999'
E
E         - 999
E         + 226

test_auth_manager.py:374: AssertionError


================== 12 passed, 3 xfailed, 1 warning in 39.46s ==================
```

```
test_auth_manager.py::TestIntegrationSchedule::test_scrape_schedule_structure PASSED [100%]
Process finished with exit code 0
```